Olist - Brazilian E-Commerce Platform

Strategic Audit (2016–2018)

Does the platform's **rapid volume growth** outpace its logistical infrastructure?

Is the long-term growth **sustainable**, or rather **dependent** on a highly concentrated group of Market Leaders and High-Yield product categories?

PostgreSQL python git GitHub

BY DOMINIKA DRĄŻYK
MARCH, 2026

Introduction

How much can we learn from **a major Brazilian marketplace** during its primary expansion phase?

This comprehensive audit extracts actionable business intelligence from a dataset of nearly 100,000 orders.

PRACTICAL BUSINESS QUESTIONS

- How does the merchant structure **impact operational reliability?**
- Is extreme revenue concentration **threatening the platform's stability?**
- How significant is the variance between delivery estimates and **actual logistics performance?**
- What does the current momentum say about the **platform's overall scalability?**

How much can we learn from **a major Brazilian marketplace** during its primary expansion phase?

This comprehensive audit extracts actionable business intelligence from a dataset of nearly 100,000 orders.

PRACTICAL BUSINESS QUESTIONS

- How does the merchant structure **impact operational reliability?**
- Is extreme revenue concentration **threatening the platform's stability?**
- How significant is the variance between delivery estimates and **actual logistics performance?**
- What does the current momentum say about the **platform's overall scalability?**

Data source:

Olist E-Commerce Public Dataset via. Kaggle

Authored by A. Sionek (2018) [link]

{ olist_customers_dataset.csv }
{ olist_geolocation_dataset.csv }
{ olist_order_items_dataset.csv }
{ olist_order_payments_dataset.csv }
{ olist_order_reviews_dataset.csv }
{ olist_orders_dataset.csv }
{ olist_products_dataset.csv }
{ olist_sellers_dataset.csv }
{ product_category_name_translation.csv }

publicly available

structured, but requires pre-processing

Approach

Phase 1: Pre-Staging Pipeline

Step 1: Unicode normalization & encoding correction

Language: Python (*pandas*, *unidecode*, *pathlib*)

Code: *ecommerce_converter.py*

Phase 2: Relational ETL Process

Step 2: Staging tables (raw data ingestion)

Step 3: DQ checks (null handling, anomaly detection)

Step 4: Idempotent load (standardizing data types, cleaning)

Step 5: Relationships, indexes, foreign keys (schema optimization)

Language: SQL (*PostgreSQL*)

Environment: pgAdmin

Code: *ecommerce_ETL.sql*

Example – encoding correction

```python
# Fallback mechanism to handle common encoding variations in the Olist dataset
encodings = ['utf-8', 'latin1', 'iso-8859-1', 'cp1252']
df = None

for enc in encodings:
    try:
        df = pd.read_csv(input_file, encoding=enc, low_memory=False)
        break
    except (UnicodeDecodeError, LookupError):
        continue

if df is None:
    raise ValueError(f"Could not read file: {input_file}")

# Isolate text-based columns for transliteration (removing accents/special chars)
text_columns = [col for col in df.columns if df[col].dtype == 'object']
for col in text_columns:
    # unidecode ensures 'São Paulo' becomes 'Sao Paulo' for consistent SQL indexing
    df[col] = df[col].apply(lambda x: unidecode.unidecode(str(x)) if pd.notna(x) else x)
# Save as standardized UTF-8 without index to maintain schema alignment
df.to_csv(output_file, index=False, encoding='utf-8')
```

SOLUTION

The script iteratively tests multiple encodings to ingest raw data without read errors. Then, it applies ASCII transliteration to neutralize Brazilian Portuguese characters, outputting a standardized UTF-8.

Phase 1: Pre-Staging Pipeline

Step 1: Unicode normalization & encoding correction

Language: Python (*pandas*, *unidecode*, *pathlib*)

Code: *ecommerce_converter.py*

Phase 2: Relational ETL Process

Step 2: Staging tables (raw data ingestion)

Step 3: DQ checks (null handling, anomaly detection)

Step 4: Idempotent load (standardizing data types, cleaning)

Step 5: Relationships, indexes, foreign keys (schema optimization)

Language: SQL (*PostgreSQL*)

Environment: pgAdmin

Code: *ecommerce_ETL.sql*

Example – raw data ingestion

```sql
DROP TABLE IF EXISTS staging_products;
CREATE TABLE staging_products (
  staging_id                  text PRIMARY KEY DEFAULT gen_random_uuid(),
                               --> Generating staging key
  loaded_file                 text DEFAULT 'olist_products_dataset_conv.csv',
                               --> Manually inserting filename
  product_id                  text,
                               --> Natural key (alphanumeric)
  product_category_name       text,
                               --> Product category (may be NULL)
  product_name_length         text,
                               --> Character count of product name
  product_description_length text,
                               --> Character count of description
  product_photos_qty          text,
                               --> Number of product photos
  product_weight_g            text,
                               --> Weight in grams
  product_length_cm           text,
                               --> Length dimension in cm
  product_height_cm           text,
                               --> Height dimension in cm
  product_width_cm            text,
                               --> Width dimension in cm
  loaded_at                   timestamptz DEFAULT now()
                               --> Audit timestamp
);
```

SOLUTION

By defining all source fields as text, the staging phase prevents premature type-casting errors during the initial .csv ingestion. For traceability, I added automated audit metadata (unique identifiers, timestamps, source file names).

Phase 1: Pre-Staging Pipeline

Step 1: Unicode normalization & encoding correction

Language: Python (*pandas*, *unidecode*, *pathlib*)

Code: *ecommerce_converter.py*

Phase 2: Relational ETL Process

Step 2: Staging tables (raw data ingestion)

Step 3: DQ checks (null handling, anomaly detection)

Step 4: Idempotent load (standardizing data types, cleaning)

Step 5: Relationships, indexes, foreign keys (schema optimization)

Language: SQL (*PostgreSQL*)

Environment: pgAdmin

Code: *ecommerce_ETL.sql*

Example – null handling

```sql
SELECT
  COUNT(*) AS total_rows,
  -- Calculate missing percentage for each field
  ROUND(100.0 * SUM(CASE WHEN product_id IS NULL
        OR trim(product_id) = '' THEN 1 ELSE 0 END) / COUNT(*), 3)
  ROUND(100.0 * SUM(CASE WHEN product_category_name IS NULL
        OR trim(product_category_name) = '' THEN 1 ELSE 0 END) / COUNT(*), 3)
  ROUND(100.0 * SUM(CASE WHEN product_name_length IS NULL
        OR trim(product_name_length) = '' THEN 1 ELSE 0 END) / COUNT(*), 3)
  ROUND(100.0 * SUM(CASE WHEN product_description_length IS NULL
        OR trim(product_description_length) = '' THEN 1 ELSE 0 END) / COUNT(*), 3)
  ROUND(100.0 * SUM(CASE WHEN product_photos_qty IS NULL
        OR trim(product_photos_qty) = '' THEN 1 ELSE 0 END) / COUNT(*), 3)
  ROUND(100.0 * SUM(CASE WHEN product_weight_g IS NULL
        OR trim(product_weight_g) = '' THEN 1 ELSE 0 END) / COUNT(*), 3)
  ROUND(100.0 * SUM(CASE WHEN product_length_cm IS NULL
        OR trim(product_length_cm) = '' THEN 1 ELSE 0 END) / COUNT(*), 3)
  ROUND(100.0 * SUM(CASE WHEN product_height_cm IS NULL
        OR trim(product_height_cm) = '' THEN 1 ELSE 0 END) / COUNT(*), 3)
  ROUND(100.0 * SUM(CASE WHEN product_width_cm IS NULL
        OR trim(product_width_cm) = '' THEN 1 ELSE 0 END) / COUNT(*), 3)
FROM staging_products;
```

SOLUTION

This query calculates the percentage of NULL or empty values for every field, identifying critical gaps before any structural transformations are applied.

Phase 1: Pre-Staging Pipeline

Step 1: Unicode normalization & encoding correction

Language: Python (*pandas*, *unidecode*, *pathlib*)

Code: *ecommerce_converter.py*

Phase 2: Relational ETL Process

Step 2: Staging tables (raw data ingestion)

Step 3: DQ checks (null handling, anomaly detection)

Step 4: Idempotent load (standardizing data types, cleaning)

Step 5: Relationships, indexes, foreign keys (schema optimization)

Language: SQL (*PostgreSQL*)

Environment: pgAdmin

Code: *ecommerce_ETL.sql*

Example – standardizing

```sql
WITH cleaned AS (
  SELECT
    staging_id,
    NULLIF(trim(product_id), '') AS product_id,
    NULLIF(trim(product_category_name), '') AS product_category,
    CASE WHEN product_name_length ~ '^\d+(\.\d+)?$'
      THEN product_name_length::numeric ELSE NULL END AS name_length,
    CASE WHEN product_description_length ~ '^\d+(\.\d+)?$'
      THEN product_description_length::numeric ELSE NULL END AS description_length,
    CASE WHEN product_photos_qty ~ '^\d+(\.\d+)?$'
      THEN product_photos_qty::numeric ELSE NULL END AS photos_qty,
    CASE WHEN product_weight_g ~ '^\d+(\.\d+)?$'
      THEN product_weight_g::numeric ELSE NULL END AS weight_g,
    CASE WHEN product_length_cm ~ '^\d+(\.\d+)?$'
      THEN product_length_cm::numeric ELSE NULL END AS length_cm,
    CASE WHEN product_height_cm ~ '^\d+(\.\d+)?$'
      THEN product_height_cm::numeric ELSE NULL END AS height_cm,
    CASE WHEN product_width_cm ~ '^\d+(\.\d+)?$'
      THEN product_width_cm::numeric ELSE NULL END AS width_cm,
    loaded_at,
    loaded_file
  FROM staging_products
  WHERE NULLIF(trim(product_id),'') IS NOT NULL  -- Exclude invalid records
),
```

SOLUTION

The CTE utilizes regex pattern matching to validate numeric strings before casting them to their proper types, trimming whitespace and converting empty strings into NULL values.

Phase 1: Pre-Staging Pipeline

Step 1: Unicode normalization & encoding correction

Language: Python (*pandas*, *unidecode*, *pathlib*)

Code: *ecommerce_converter.py*

Phase 2: Relational ETL Process

Step 2: Staging tables (raw data ingestion)

Step 3: DQ checks (null handling, anomaly detection)

Step 4: Idempotent load (standardizing data types, cleaning)

Step 5: Relationships, indexes, foreign keys (schema optimization)

Language: SQL (*PostgreSQL*)

Environment: pgAdmin

Code: *ecommerce_ETL.sql*

Example – schema optimization

```sql
ALTER TABLE fact_order_items
  ADD CONSTRAINT fk_foi_order_sk
      FOREIGN KEY (order_sk) REFERENCES dim_orders(order_sk),
  ADD CONSTRAINT fk_foi_product_sk
      FOREIGN KEY (product_sk) REFERENCES dim_products(product_sk),
  ADD CONSTRAINT fk_foi_seller_sk
      FOREIGN KEY (seller_sk) REFERENCES dim_sellers(seller_sk);
```

```sql
CREATE INDEX IF NOT EXISTS
    idx_foi_order_sk ON fact_order_items(order_sk);
CREATE INDEX IF NOT EXISTS
    idx_foi_product_sk ON fact_order_items(product_sk);
CREATE INDEX IF NOT EXISTS
    idx_foi_seller_sk ON fact_order_items(seller_sk);
CREATE INDEX IF NOT EXISTS
    idx_foi_order_id ON fact_order_items(order_id);
```

SOLUTION

The query enforces constraints (foreign keys and their references) and indexes to establish a formal star-schema architecture.

Phase 1: Pre-Staging Pipeline

Step 1: Unicode normalization & encoding correction

Language: Python (*pandas*, *unidecode*, *pathlib*)

Code: *ecommerce_converter.py*

Phase 2: Relational ETL Process

Step 2: Staging tables (raw data ingestion)

Step 3: DQ checks (null handling, anomaly detection)

Step 4: Idempotent load (standardizing data types, cleaning)

Step 5: Relationships, indexes, foreign keys (schema optimization)

Language: SQL (*PostgreSQL*)

Environment: pgAdmin

Code: *ecommerce_ETL.sql*

Intermediate results

{ olist_customers_dataset_conv.csv } …

{ olist_geolocation_dataset_conv.csv }
{ olist_order_items_dataset_conv.csv }
{ olist_order_payments_dataset_conv.csv }
{ olist_order_reviews_dataset_conv.csv }
{ olist_orders_dataset_conv.csv }
{ olist_products_dataset_conv.csv }
{ olist_sellers_dataset_conv.csv }
{ product_category_name_translation_conv.csv }

Olist datasets converted (pre-staging) before loading;

ecommerce_converter.py

Python pre-staging code that can be reused in future;

ecommerce_ETL.sql

PostgreSQL ETL code that can be reused in future;

Approach

{ olist_customers_dataset_conv.csv } ...

{ olist_geolocation_dataset_conv.csv }
{ olist_order_items_dataset_conv.csv }
{ olist_order_payments_dataset_conv.csv }
{ olist_order_reviews_dataset_conv.csv }
{ olist_orders_dataset_conv.csv }
{ olist_products_dataset_conv.csv }
{ olist_sellers_dataset_conv.csv }
{ product_category_name_translation_conv.csv }

Olist datasets converted (pre-staging) before loading;

ecommerce_converter.py

Python pre-staging code that can be reused in future;

ecommerce_ETL.sql

PostgreSQL ETL code that can be reused in future;

Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Example – dynamic thresholding

```sql
), market_ldr_thresholds AS(
    SELECT DISTINCT
        PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY late_delivery_rate) AS vol_p25,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY late_delivery_rate) AS vol_p75
    FROM seller_late_delivery_rate
)
SELECT
    sldr.seller_sk AS seller_id,
    sldr.late_delivery_rate,
    mldr.vol_p25,
    mldr.vol_p75,
    CASE WHEN sldr.late_delivery_rate > mldr.vol_p75 THEN '3 - high risk'
         WHEN sldr.late_delivery_rate <= mldr.vol_p25 THEN '1 - low risk'
         ELSE '2 - medium risk' END AS reliability_tier
FROM
    seller_late_delivery_rate sldr,
    market_ldr_thresholds mldr
ORDER BY sldr.seller_sk;
```


Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

SOLUTION

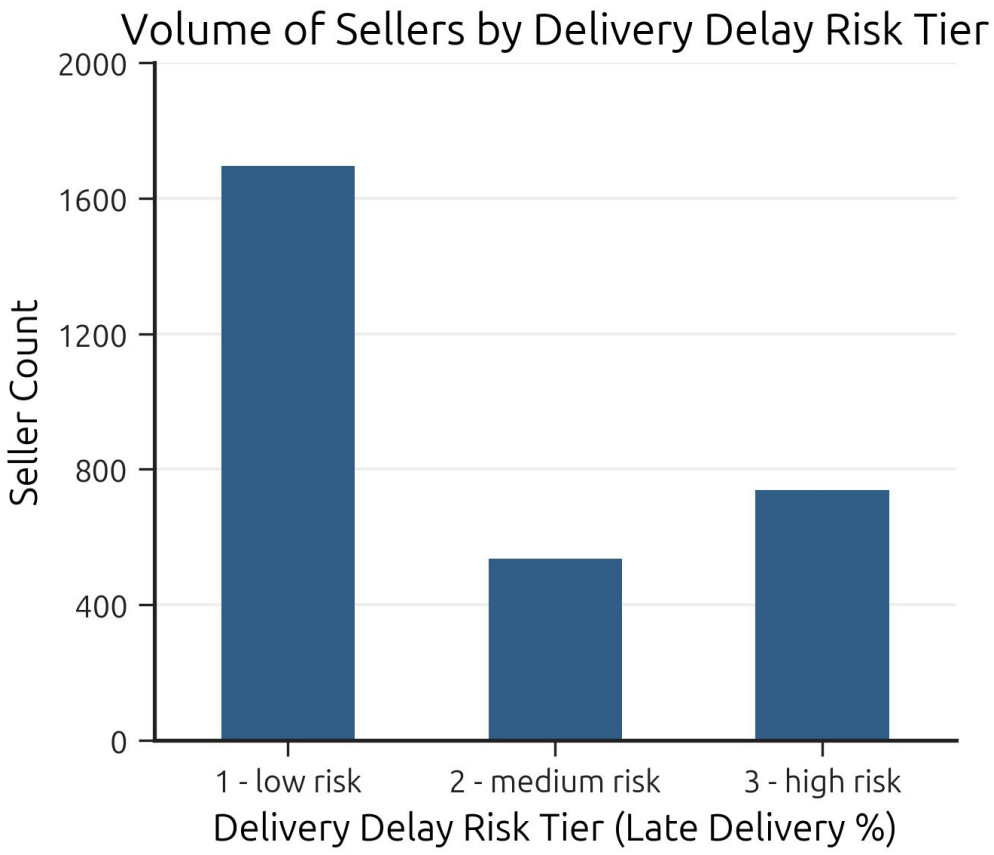
Using the Interquartile Range (IQR) I establish self-adjusting thresholds at the 25th and 75th percentiles to objectively categorize merchants into reliability tiers. This approach avoids hardcoded KPIs, allowing the business to tier based on current market reality.

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Example – dynamic thresholding



Business Insight

The prevalence of Low Risk sellers suggests that the platform's delivery infrastructure and the logic of estimating delivery time are well-aligned for the majority of merchants.

Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Example – complex visualizations

```
fig, ax = plt.subplots(figsize=(5, 5))

pl = sns.regplot(data = df_q211, x = 'total_units', y = 'total_revenue',
                 color = full_palette[0], ax = ax);

slope, intercept, r_val, p_val, std_err = stats.linregress(
                df_q211['total_units'], df_q211['total_revenue'])

y_pred = intercept + slope * df_q211['total_units']
residuals = df_q211['total_revenue'] - y_pred
see = np.sqrt(np.sum(residuals**2) / (len(df_q211) - 2))
threshold = y_pred + see
outliers = df_q211[df_q211['total_revenue'] > threshold]

r_squared = r_val**2
ax.text(0.05, 0.95, f'$R^2 = {r_squared:.2f}$',
        transform=ax.transAxes, verticalalignment='top', fontsize=10,
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.5))

for i, row in outliers.iterrows():
    ax.annotate(
        row['product_category'], xy = (row['total_units'], row['total_revenue']),
        xytext = (3, 6), textcoords = 'offset points', fontsize = 9, rotation = 45,
        bbox = dict(boxstyle = "square,pad=0.075",fc = "white", ec = "white", lw = 1))
```

SOLUTION

Using a regression-based anomaly detection, I identify product categories that significantly outperform the market baseline. The code establishes a dynamic threshold to isolate positive outliers and incorporates an R2 coefficient to validate the model's fit.

Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

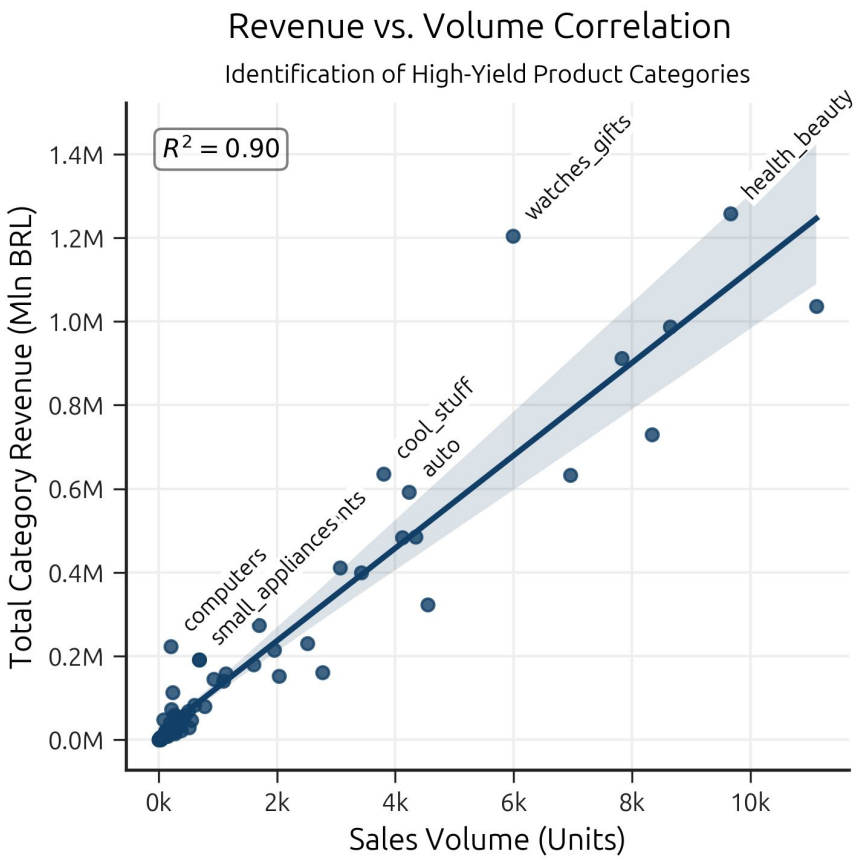
Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Example – complex visualizations



Business Insight

The Watches_Gifts is the standout performer - it produces nearly as much revenue as Health_Beauty but requires significantly less logistical throughput. This category represents the ideal balance of high margin and manageable volume.

Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Example – window functions

```sql
), category_segmentation AS (
    SELECT
        CASE
            WHEN ROW_NUMBER() OVER (ORDER BY total_revenue DESC) <= 5
            THEN category
            ELSE 'other'
        END AS market_segment,
        total_revenue,
        units_sold
    FROM category_metrics
)
SELECT
    market_segment,
    SUM(units_sold) AS total_units,
    SUM(total_revenue) AS total_revenue,
    SUM(total_revenue) * 1.0 / SUM(SUM(total_revenue)) OVER() AS perc_in_revenue
FROM category_segmentation
GROUP BY market_segment
ORDER BY total_revenue DESC;
```


Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

SOLUTION

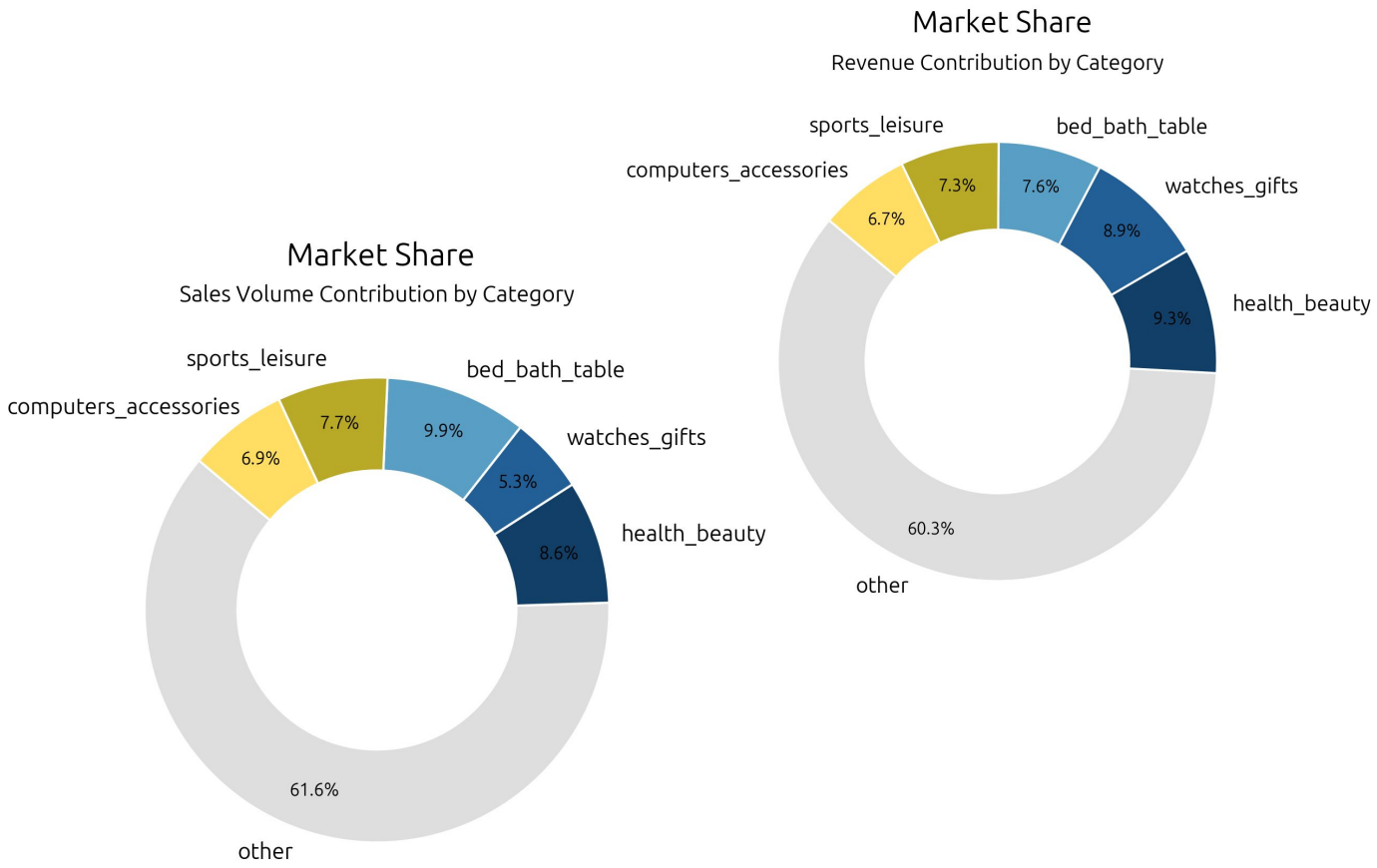
In this CTE, I use window function to isolate the top five revenue-generating categories from the market's residual (other). Then I calculate their share in combined platform's revenue (perc_in_revenue).

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Example – window functions



Business Insight

The high revenue-to-volume ratio of Watches_Gifts suggests that marketing spend in this category is highly efficient, yielding more dollars per unit shipped than any other top category.

Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

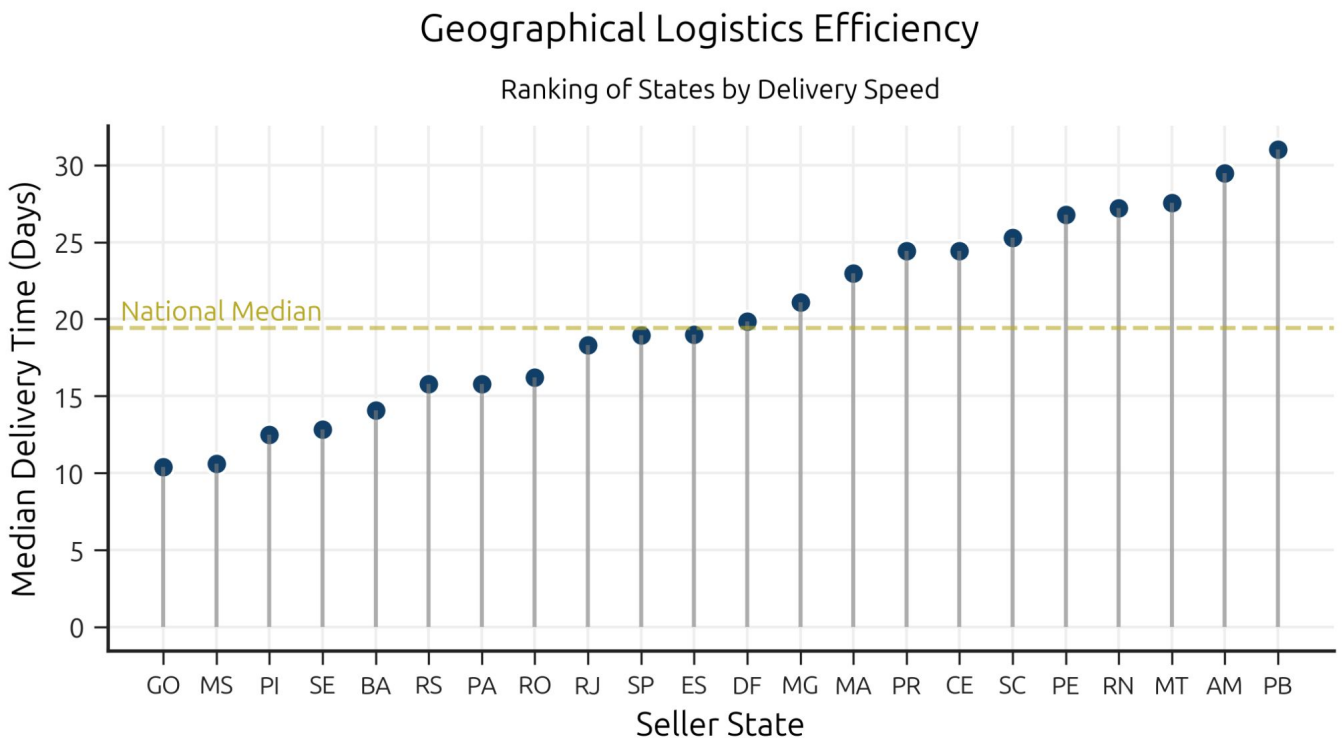
Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Example – complex visualizations



Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

Business Insight

The platform should adjust the delivery time estimate to specific regions: for an AM, the delivery time estimate needs to be significantly extended, while for a GO, it could be shortened to drive competitiveness.

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Example – rolling measures

```sql
), rolling_monthly_revenue AS (
    SELECT
        market_segment,
        monthly,
        this_month_revenue,
        LAG(this_month_revenue, 1) OVER (
            PARTITION BY market_segment ORDER BY monthly)
            AS prior_month_revenue,
        AVG(this_month_revenue) OVER (
            PARTITION BY market_segment ORDER BY monthly ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
            AS T3M_revenue
    FROM monthly_revenue
)
SELECT
    market_segment,
    monthly,
    this_month_revenue,
    prior_month_revenue,
    T3M_revenue,
    ROUND(((this_month_revenue - prior_month_revenue) / NULLIF(prior_month_revenue, 0) * 100), 2)
        AS mom_growth_pct
FROM rolling_monthly_revenue
WHERE prior_month_revenue IS NOT NULL
ORDER BY market_segment, monthly;
```

Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

SOLUTION

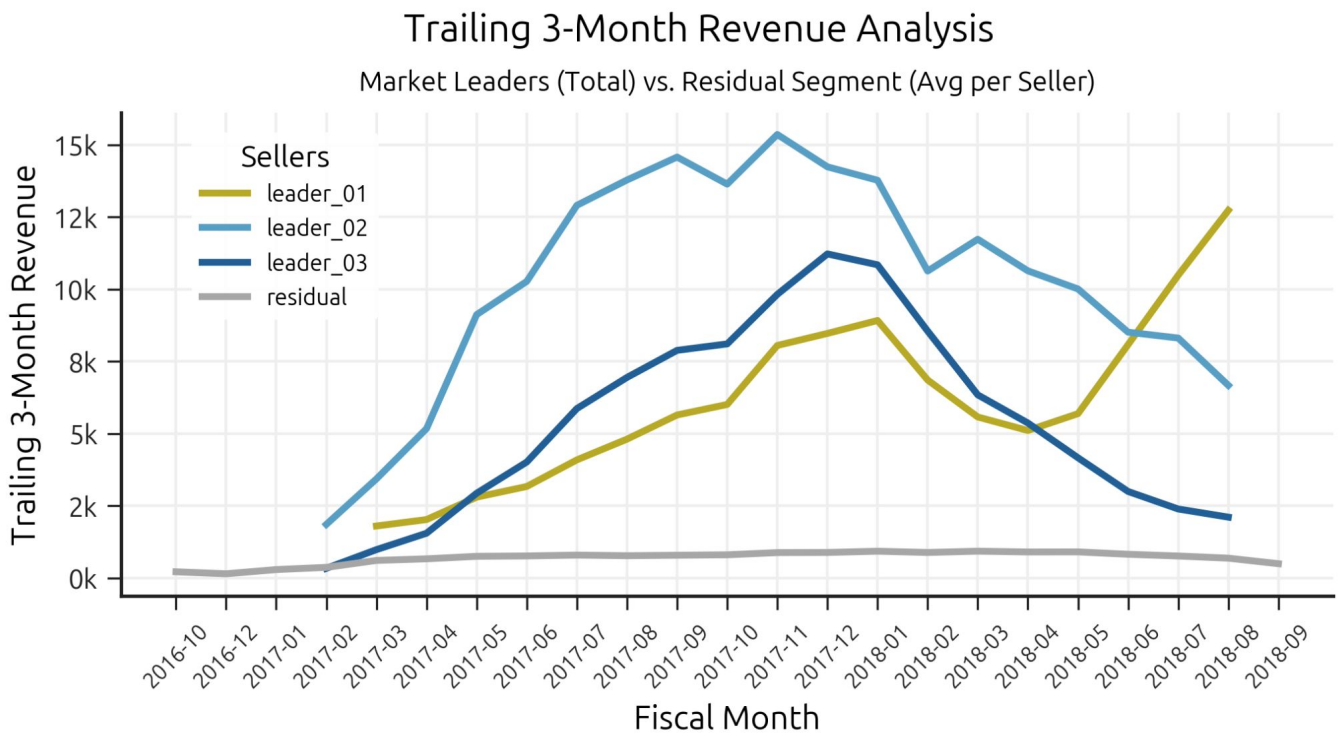
Using window functions, I implement a time-series smoothing and growth analysis to evaluate the platform's revenue momentum across different market segments. I also include Zero-Division Protection (NULIFF()) in calculations.

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Example – rolling measures



Q1: Seller Performance & Delivery Delays Over Time

Q1.1 Quarterly Order Volume by Seller Performance Tier
Q1.2 Seller Distribution across Delivery Delay Tiers
Q1.3 Trailing 6 Months Late Delivery Rate
Q1.4 Dynamic Rank of Seller Delivery Delay Risk

Q2: Product Category Profitability & Price Sensitivity

Q2.1 Category Portfolio: Revenue, Volume and Price
Q2.2 Global and Internal Revenue Share by Price Tier

Q3: Geographic Impact on Delivery Performance

Q3.1 Median Delivery Time by Geographical State
Q3.2 Delivery Reliability by Geographical State
Q3.3 Revenue at Risk Due to Late Deliveries

Q4: Market Growth Trajectory & Revenue Momentum

Q4.1 Monthly Revenue of Market Leaders
Q4.2 MoM Growth Rate of Market Leaders
Q4.3 Trailing 3 Month Revenue of Market Leaders

Business Insight

The divergence between Leader 01 and the other segments in 2018 suggests a fundamental difference in operational strategy. The "Residual Stagnation" is a significant strategic concern; it suggests a lack of "growth engine" for small-to-medium sellers.

Language: Python {*pandas*, *matplotlib*, *seaborn*, *SQLAlchemy*}

Environment: Jupyter Notebook

Code: *ecommerce_analysis.ipynb*

Summary – key findings

The platform's growth was defined by **high volatility** and **structural concentration**, failing to achieve the sustained, linear expansion of the broader seller base required for long-term ecosystem health.

Core vulnerabilities:

- **Revenue Polarisation:** few High-Yield product categories and Market Leaders paired with the stagnation of the Residual segment.
- **Logistical Disconnect:** high order volume does not correlate with operational excellence. Market Leaders are highly susceptible to supply chain friction.
- **Regional Asymmetry:** deeply fragmented logistical landscape (revenue-dominant São Paulo versus low-volume Amazonas).

Summary – key findings

The platform's growth was defined by **high volatility** and **structural concentration**, failing to achieve the sustained, linear expansion of the broader seller base required for long-term ecosystem health.

Core vulnerabilities:

- **Revenue Polarisation:** few High-Yield product categories and Market Leaders paired with the stagnation of the Residual segment.
- **Logistical Disconnect:** high order volume does not correlate with operational excellence. Market Leaders are highly susceptible to supply chain friction.
- **Regional Asymmetry:** deeply fragmented logistical landscape (revenue-dominant São Paulo versus low-volume Amazonas).

Business Insight:

- The platform must introduce **a growth engine** for middle-class sellers.
- Growth strategies of Leaders must be paired with **mandatory logistical upgrades**.
- Logistics recovery efforts must be **prioritized in highest-density revenue streams** first.

Summary – analysis limitations

- *The 100,000-order snapshot is a **historical "frozen" dataset** (2016–2018) and may not reflect recent post-pandemic shifts in Brazilian e-commerce behavior or logistics;*

- *Models do not account for **external macroeconomic factors** (e.g., BRL exchange rate volatility, national strikes, changes in postal service pricing) during the window;*

- *The SQL views treat **cancelled orders** as lost revenue but cannot distinguish between seller-fault cancellations and customer-driven returns;*

- *The T3M and MoM growth models assume a degree of continuity that **may overlook micro-seasonal events** (e.g., local holidays).*

Important note:

Offered insights are entirely focused on internal marketplace dynamics and are not based on academic knowledge of Brazilian macroeconomics.

My aim was to demonstrate the ability to architect data, design robust ETL processes, and extract actionable business intelligence from complex relational datasets.

Final results

- *The 100,000-order snapshot is a **historical "frozen" dataset** (2016–2018) and may not reflect recent post-pandemic shifts in Brazilian e-commerce behavior or logistics;*
- *Models do not account for **external macroeconomic factors** (e.g., BRL exchange rate volatility, national strikes, changes in postal service pricing) during the window;*
- *The SQL views treat **cancelled orders** as lost revenue but cannot distinguish between seller-fault cancellations and customer-driven returns;*
- *The T3M and MoM growth models assume a degree of continuity that **may overlook micro-seasonal events** (e.g., local holidays).*

Important note:
Offered insights are entirely focused on internal marketplace dynamics and are not based on academic knowledge of Brazilian macroeconomics.
My aim was to demonstrate the ability to architect data, design robust ETL processes, and extract actionable business intelligence from complex relational datasets.

{ olist_customers_dataset_conv.csv } …
Olist datasets converted (pre-staging) before loading;

ecommerce_converter.py
Python pre-staging code that can be reused in future;

ecommerce_ETL.sql
PostgreSQL ETL code that can be reused in future;

ecommerce_analysis.ipynb
queries and visualizations that can be reused in future;

{ int_leaders_late_delivery_count } …

{ int_market_leaders }
{ rpt_q1_leaders_l6m_late_delivery_rate }
{ rpt_q1_leaders_market_share }
{ rpt_q1_leaders_market_share_delay_risk }
{ rpt_q1_leaders_monthly_ldr_segmentation }
{ rpt_q1_seller_delivery_delay_segmentation }
{ rpt_q1_seller_quarterly_performance }
{ rpt_q2_product_category_performance }
{ rpt_q2_top_categories_in_revenue }
{ rpt_q2_top_categories_in_revenue_priceband }
{ rpt_q3_state_delivery_estimation_reliability }
{ rpt_q4_leaders_mom_growth }

a set of intermediate and report-ready SQL views;

Key skills

Data modelling & engineering

- Designing a star-schema model (fact & dimension tables) within a PostgreSQL environment.
- Creating robust one-to-many and one-to-one relationships across complex relational datasets.
- Resolving data ambiguity and category mapping using lookup and bridging tables.
- Building advanced aggregation measures (e.g., T3M smoothing, MoM growth, percentile-based thresholds).

Data visualization & storytelling

- Translating a strategic business narrative into a functional, highly readable analytical report.
- Highlighting high-impact segments (e.g., Market Leaders, high-yield categories, and at-risk regions) visually.
- Organizing metrics and visuals into purposeful analytical chapters aligned with an executive audit scenario.
- Preparing comprehensive documentation to effectively communicate methodology.
- Translating data into actionable business insights.